TECNICATURA UNIVERSITARIA EN INTELIGENCIA ARTIFICIAL (TUIA)

PROCESAMIENTO DE IMÁGENES(PDI)

Grupo Nro 1:

ELIZONDO RIOS, Iñaki Cruz Legajo: E-1260/2
JUAREZ, Leandro Santiago Legajo: J-0633/5
TORINO, Facundo Legajo: T-3132/1

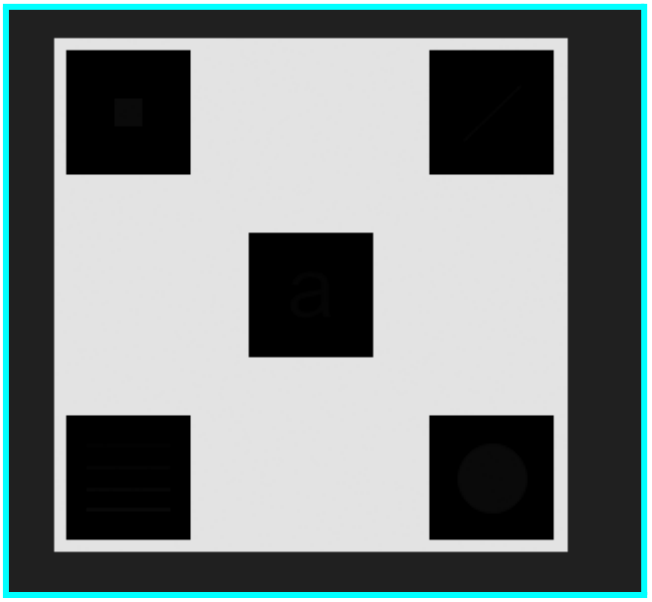
AÑO 2026 - 1er CUATRIMESTRE-

PRELIMINAR: los archivos de resolución pueden encontrarse en
https://github.com/Ftedp/PDI-TP1

PRESENTACIÓN DEL PROBLEMA 1 :

La consigna del trabajo práctico, solicita encontrar imágenes ocultas dentro de la siguiente imagen. De la consigna se hace referencia que debe utilizarse una estrategia de ecualización local**.**

imagen original



FUNDAMENTACIÓN TEÓRICA PARA LA SOLUCIÓN:

Sabemos por el planteo que hay imágenes ocultas, dentro de la imagen, las mismas podrían estar tanto tanto en las zonas oscuras como en las zonas claras, pero al ser una imagen tan contrastada (zonas netamente oscuras, y zonas netamente claras agrupadas) una ecualización global no podría dar buenos resultados, en cambio utilizar una ecualización local en donde se aplique una ventana deslizante donde la influencia de cada pixel a analizar está dado por él y por los vecinos próximos (definiendo distintas áreas de superficie para la ventana deslizante) esperamos que obtenga mejores resultados, ya que permitirá diferenciar zonas que a priori tienen intensidades muy parecidas pero distintas, que es justo lo que queremos resaltar.

IMPLEMENTACIÓN DE LA SOLUCIÓN:

Para implementar la ecualización local se desarrolló una función que recibe como parámetros:

- la imagen en escala de grises (img)
- el tamaño de la ventana de procesamiento (parámetros M y N)

```
def ecualizacion_local(img, M, N):
```
,

Si bien lo habitual es que las ventanas sean simétricas al mismo ancho y alto , dejamos parámetros separados por si quisiera experimentar con tamaños distintos.

Un tema a resolver fue que pasa cuando parte de la ventana cae fuera de los bordes de la imagen suministrada, para eso se utilizó la función cv2.copyMakeBorder, en donde se le indica cuántos píxeles agregar en esa zona,y que valores asignarles, terminamos resolviendo por utilizar el método BORDER_REPLICATE que lo que hace es repetir los valores del pixel más cercano perteneciente a la imagen original, en este paso también se determina hasta cuantos píxeles se agregaran a cada borde

```
img_pad = cv2.copyMakeBorder(img, top, bottom, left, right, cv2.BORDER_REPLICATE)
```

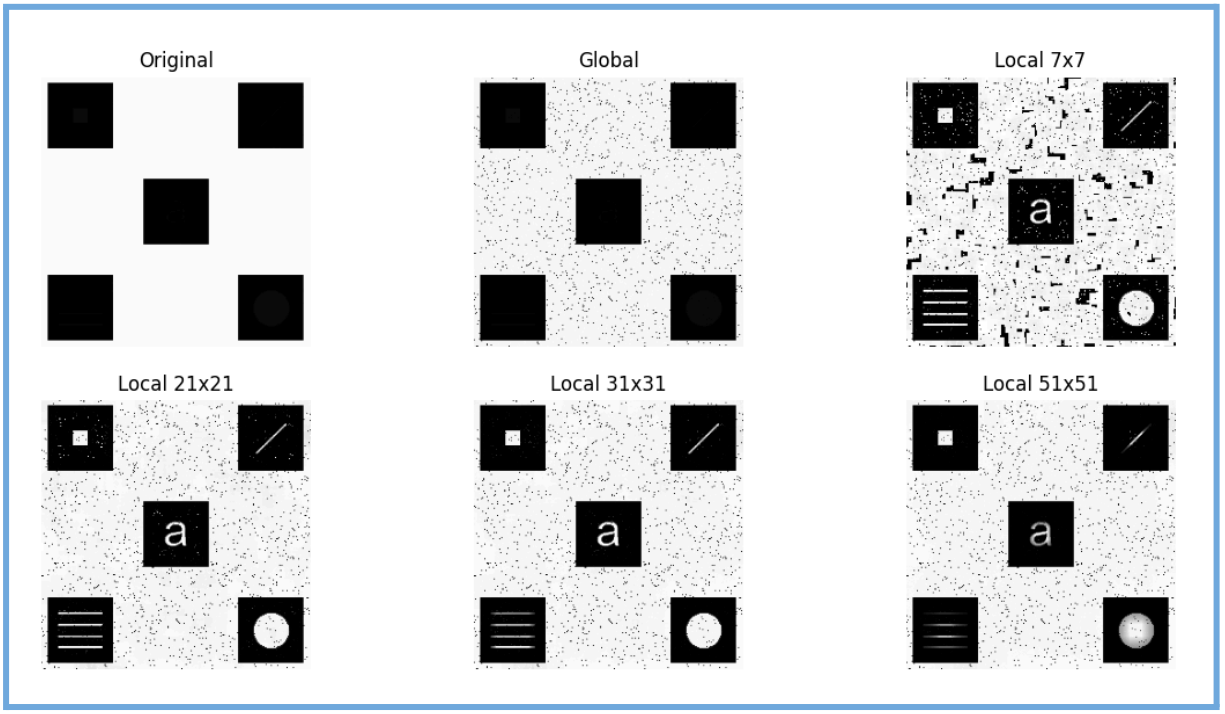
más adelante en la función realizamos la iteración

```
#creo imagen de salida
img_out = np.zeros(img.shape, dtype=np.uint8)


for i in range(filas):
    for j in range(columnas):

        # Extraer la ventana MxN centrada en el píxel (i, j)
        ventana = img_pad[i : i + M, j : j + N]

        # --- Ecualizar el histograma de la ventana (subimagen) -> (CDF) Función de Distribución Acumulada  ---
        ventana_heq = cv2.equalizeHist(ventana)

        # --- Tomar el píxel central de la ventana ecualizada ---
        img_out[i, j] = ventana_heq[top, left]

return img_out
```

La idea fue recorrer la imagen pixel por pixel, para cada uno de ellos calcular la ventana (M, N) bajo la que se va a realizar la ecualización local y calculamos la ecualización utilizando `cv2.equalizeHist` , con ello logramos que al pixel de salida se le asigne el valor del píxel central de la ventana ecualizada, y realizamos eso en toda la imagen (estamos recorriendo la matriz de la imagen original en un bucle for anidado de filas y columnas para cada fila vamos recorriendo cada una de las posiciones de la columnas). Con esto logramos que cada píxel sea transformado considerando únicamente su contexto local, lo que mejora la capacidad de resaltar detalles escondidos.

En forma experimental fuimos trabajando con distintos tamaños de ventanas para ver los resultados obtenidos, también probamos realizar una ecualización global.

Resultados obtenidos con distintos tipos de ecualización, global y diversos tamaños de ventana locales.



<u>OBSERVACIONES:</u>

Mediante este procedimiento, de experimentación cambiando los tamaños de las ventanas vemos la evolución de los resultados en donde pueden distinguirse las imágenes ocultas, pero notamos que los resultados logrados tenían un tipo de ruido particular en la imagen, el denominado “salt and pepper”,

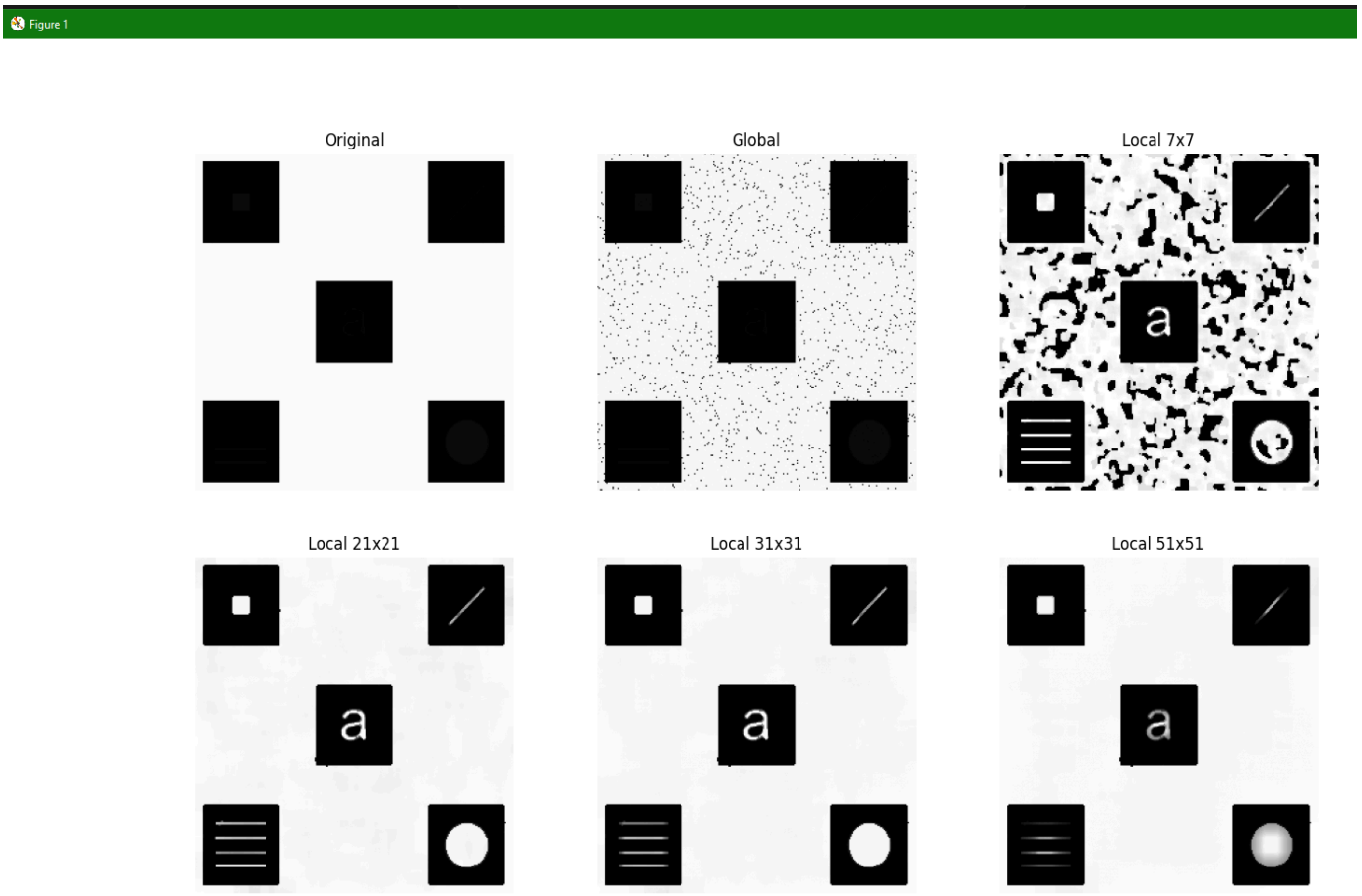
SUAVIZADO DE LA IMAGEN (Eliminando el ruido "salt an pepper"

Para intentar mejorar el ruido, decidimos utilizar un filtro de mediana (cv2.mediaBlur) partiendo de las imágenes a las que habíamos realizado la ecualización local obteniendo mejorar la imagen.

codificación para lograr la eliminación del ruido salt and pepper

```
#Filtro de la mediana para suavizar ruido
resultados = []
for M, N in ventanas:
    res = ecualizacion_local(img, M, N)
    res_filtrado = cv2.medianBlur(res, 3)
    resultados.append(res_filtrado)
```

Mostramos los resultados obtenidos para cada uno de los casos .



REFLEXIONES FINALES A PROBLEMA 1

La estrategia de ecualización local nos ha permitido descubrir las imágenes ocultas en las zonas oscuras, la técnica de suavizado nos permite mejorar aún más la presentación de la imagen reduciendo el ruido de la misma , logrando así los objetivos que se nos habían propuesto .

PRESENTACIÓN DEL PROBLEMA 2: - EXÁMENES MULTIPLE CHOISE

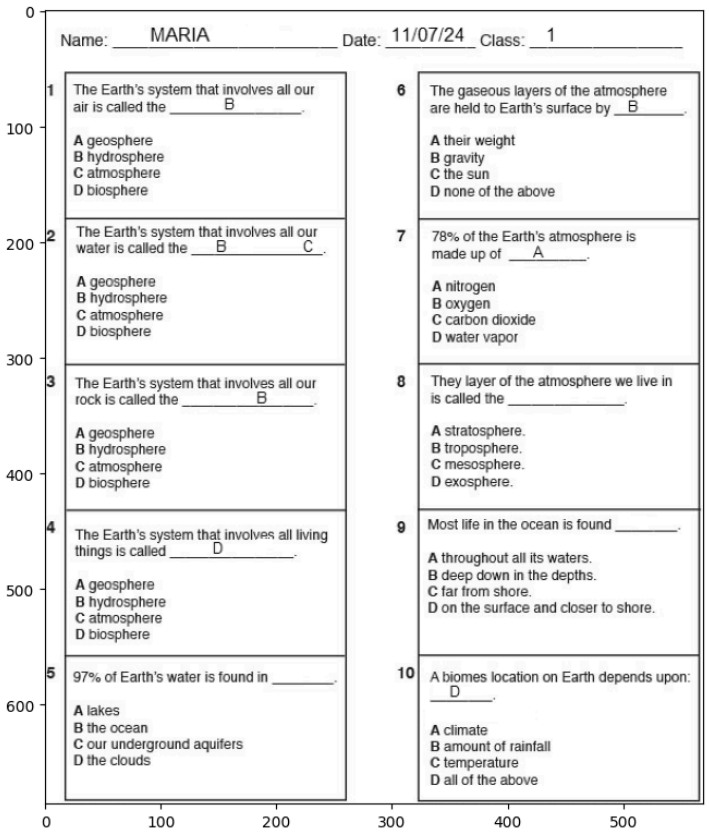
Se nos solicita que tomando por entrada un conjunto de imágenes de exámenes académicos que tienen por modalidad de “multiple choice” generar un solución para realizar la verificación de aspectos formales de los mismos, (nombre, fecha, clase) , como así también un mecanismo de verificación de la respuesta del examinado.

ABORDAJE DE LA SOLUCIÓN:

Dada la complejidad del problema y para poder ir realizando experimentaciones y tareas preparatorias para lograr una solución operativa las acciones que realizamos para llegar a la solución se documentaron en dos 2 archivos con objetivos distintos.

Exploracion_tp1_examenes.py : Tienen por objetivo dar un abordaje gradual y de los desafíos que se plantearon, buscando resolver las dificultades que se fueron presentando , su ejecución debe ser por sectores (tal cual lo hacemos habitualmente en clase) para poder ir interpretando los resultados

Captura de un exámen objeto del análisis.



Name: MARIA Date: 11/07/24 Class: 1

1 The Earth's system that involves all our air is called the B.
A geosphere
B hydrosphere
C atmosphere
D biosphere

2 The Earth's system that involves all our water is called the B C.
A geosphere
B hydrosphere
C atmosphere
D biosphere

3 The Earth's system that involves all our rock is called the B.
A geosphere
B hydrosphere
C atmosphere
D biosphere

4 The Earth's system that involves all living things is called D.
A geosphere
B hydrosphere
C atmosphere
D biosphere

5 97% of Earth's water is found in _____.
A lakes
B the ocean
C our underground aquifers
D the clouds

6 The gaseous layers of the atmosphere are held to Earth's surface by B.
A their weight
B gravity
C the sun
D none of the above

7 78% of the Earth's atmosphere is made up of A.
A nitrogen
B oxygen
C carbon dioxide
D water vapor

8 They layer of the atmosphere we live in is called the _____.
A stratosphere.
B troposphere.
C mesosphere.
D exosphere.

9 Most life in the ocean is found _____.
A throughout all its waters.
B deep down in the depths.
C far from shore.
D on the surface and closer to shore.

10 A biomes location on Earth depends upon: D.
A climate
B amount of rainfall
C temperature
D all of the above

Resolucion_tp1_examenes.py : Tiene por objetivo funcionar como un Script ordenado y de ejecución única que resuelva el problema, es decir una versión operativa para resolver la situación de la corrección sistemática de exámenes

En el presente informe analizaremos segmentos de ambos archivos concentrándose en las salidas y las estrategias utilizadas para ir resolviendo el problema y cómo llegamos a elaborar las salidas.

Ejemplo de salidas finales logradas en el script Resolucion_tp1_examenes.py diseñado para realizar una correción

```
--- Examen: TP_1/examen_2.png ---
Name:  MAL
Date:  OK
Class: OK
Pregunta  1: MAL  (detectada: B, correcta: C)
Pregunta  2: MAL  (detectada: MAL, correcta: B)
Pregunta  3: MAL  (detectada: B, correcta: A)
Pregunta  4: OK  (detectada: D, correcta: D)
Pregunta  5: MAL  (detectada: MAL, correcta: B)
Pregunta  6: OK  (detectada: B, correcta: B)
Pregunta  7: OK  (detectada: A, correcta: A)
Pregunta  8: MAL  (detectada: MAL, correcta: B)
Pregunta  9: MAL  (detectada: MAL, correcta: D)
Pregunta 10: OK  (detectada: D, correcta: D)
Resultado: DESAPROBADO (4/10 correctas)
```

IMPLEMENTACIÓN DE LA SOLUCIÓN

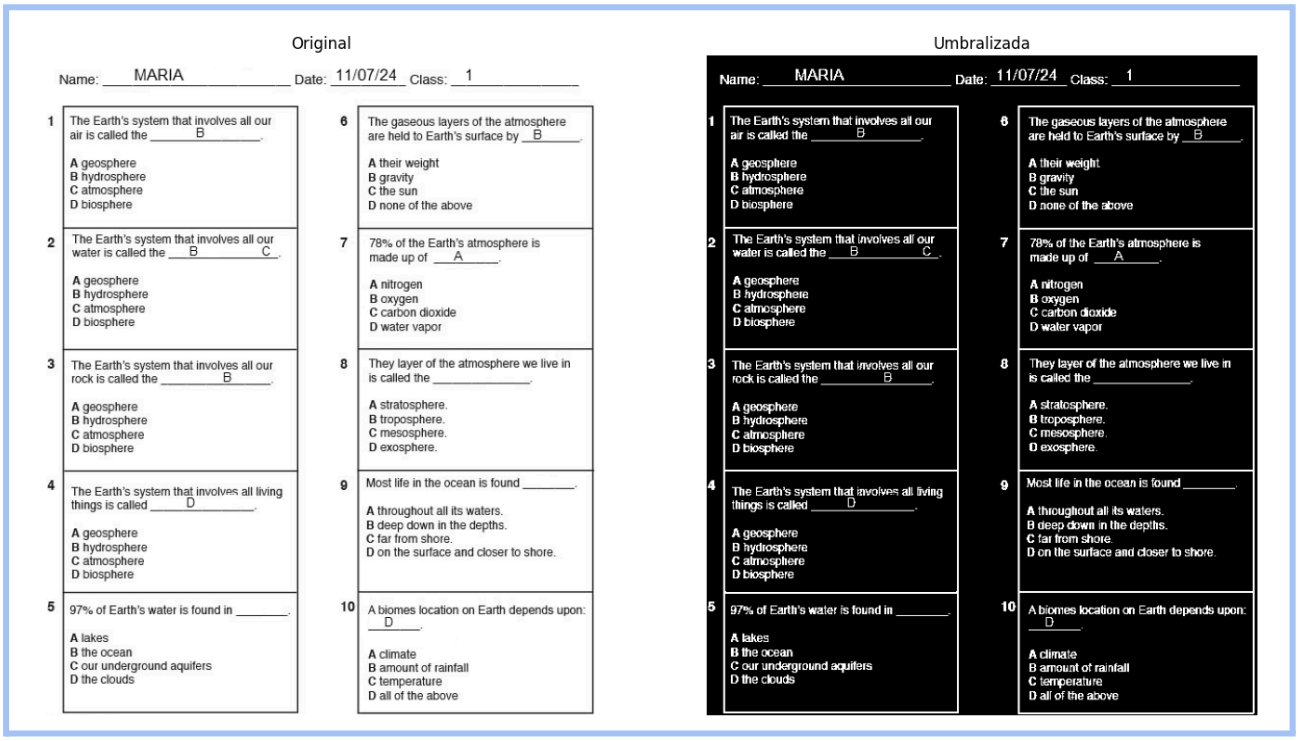
Binarización de la imagen :

Para simplificar los valores de píxeles posibles, y poder distinguir el fondo de los datos de interés, se decidió trabajar con imágenes binarizadas, lo que se logra por técnicas de umbralado

Copia de la imagen y binarización y tipo de dato

```
img_th = (img.copy() < 200).astype(np.uint8)
```

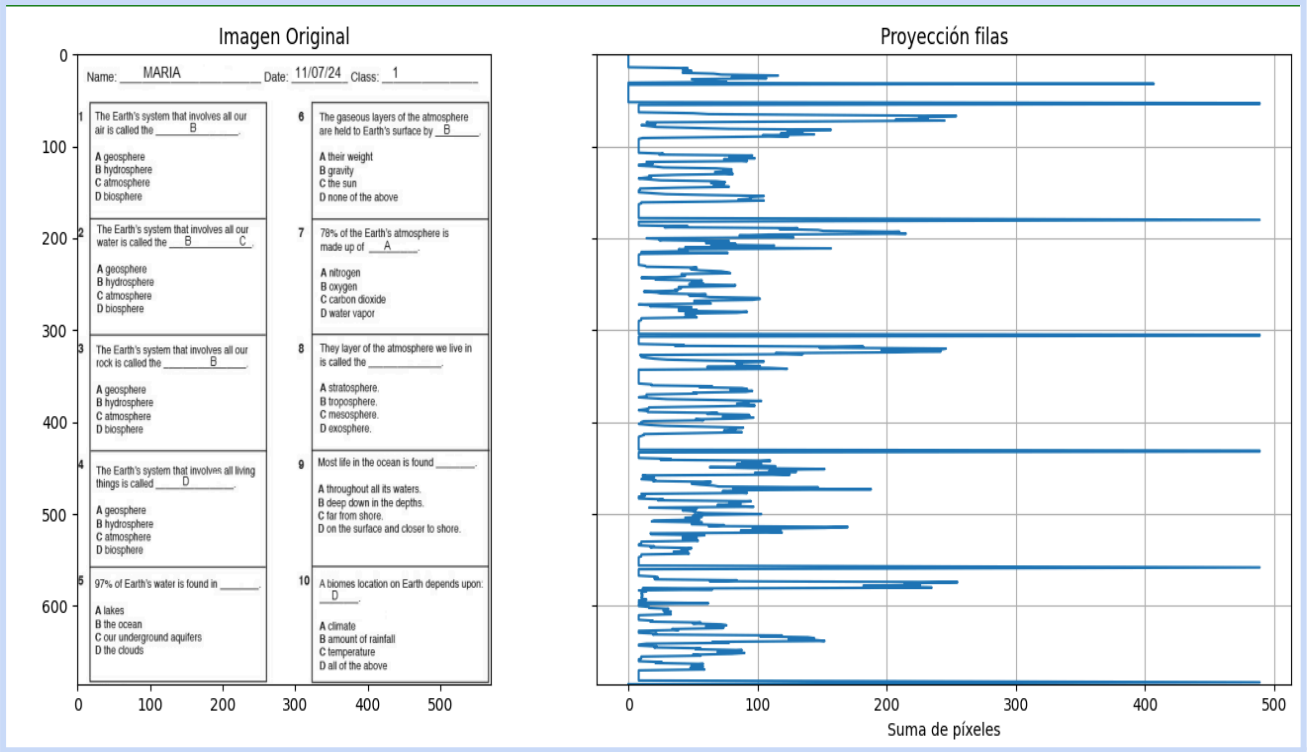
vista de la imagen original y la imagen binarizada



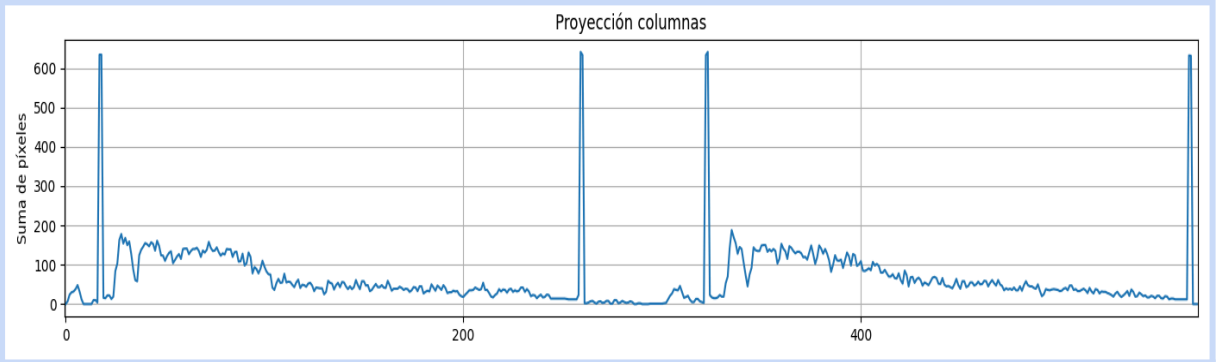
Detección de líneas

Teniendo en cuenta que estamos trabajando con una imagen binarizada, con valores 0 ó 1, contar cuál es la suma de los valores en filas y en columnas nos da una idea de cuales son las filas o las columnas más “densas”, de la observación de la imagen sabemos que los sectores del exámen, (encabezado y zona de preguntas) están limitadas por líneas rectas verticales u horizontales. Tomamos esta estrategia para detectar cuáles serían las líneas importantes para sectorizar , correlacionando los resultados de los picos obtenidos con los actores de la imagen , las filas que tengas valores más altos serán posiblemente las líneas de interés y así lo mismo con las columnas

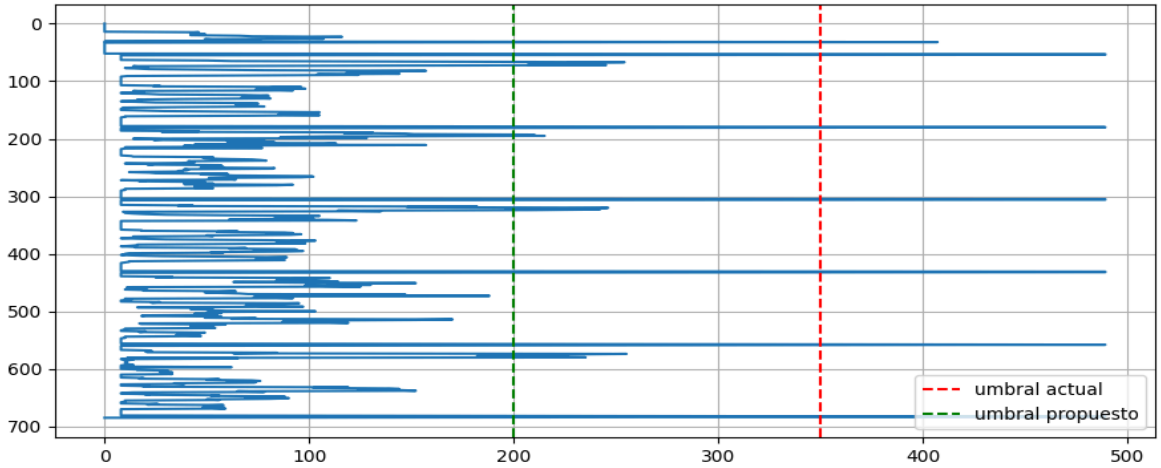
detección de líneas horizontales y verticales, y su correlación con la imagen



detección de líneas verticales



exploración de umbrales para lograr el ajuste deseado



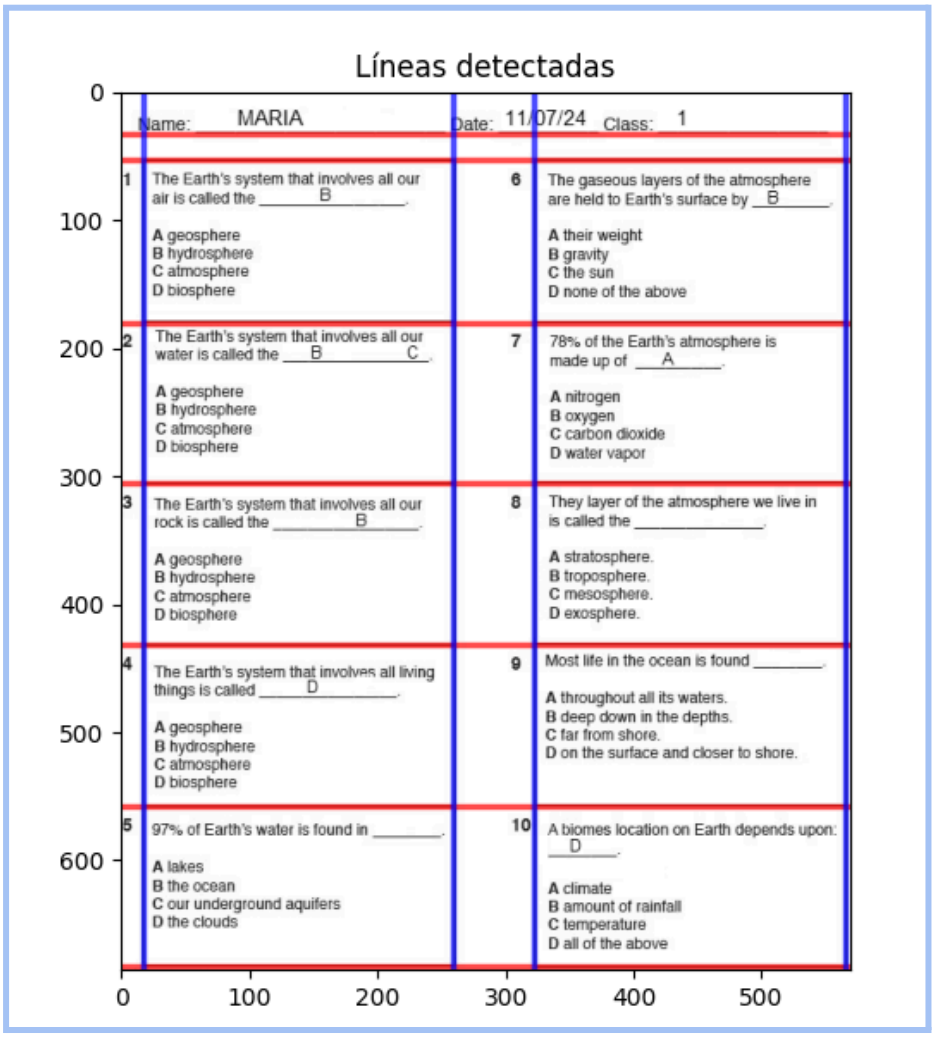
Afinando la detección de líneas:

Debido a que el mecanismo de detección por filas y columnas dio primariamente filas contiguas que tiene que ver con el grosor de la línea original ,realizamos un agrupamiento, bajo la premisa que aquellos índices detectados que son contiguos en realidad pertenecen a una misma línea, de modo experimental probamos con distintos “gap” para agruparlos hasta dar con el tamaño de agrupamiento que sirva para la corrección

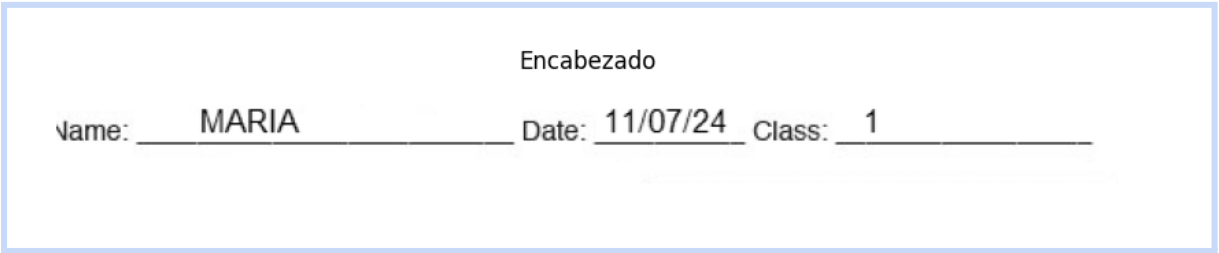
Detección de las zonas de interés

Para seguir avanzado con la solución el próximo desafío fue el partir la imagen en diversas zonas de interés, buscábamos dividir la imagen en diversas zonas, por un lado una zona correspondiente al área del encabezado , como así la zona de cada pregunta en particular por eso analizamos con las coordenadas de filas y columnas detectas hacer una sectorización de la imagen

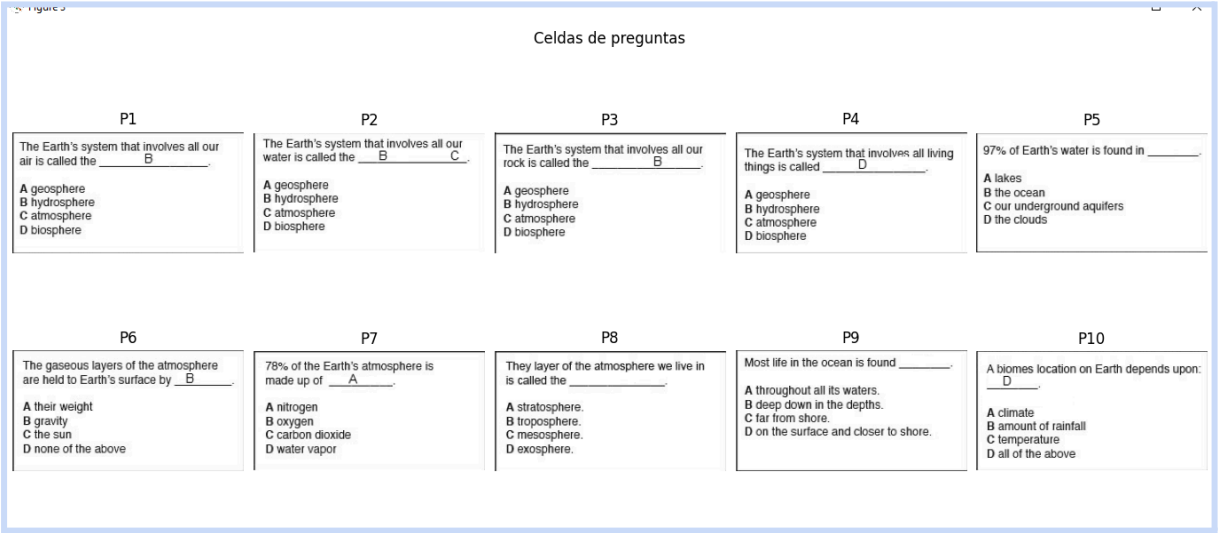
líneas horizontales(rojas) y verticales (azules) de interés para la sectorizaciòn



visualización del encabezado



Visualización de las preguntas.



Análisis de lo realizado por el alumno

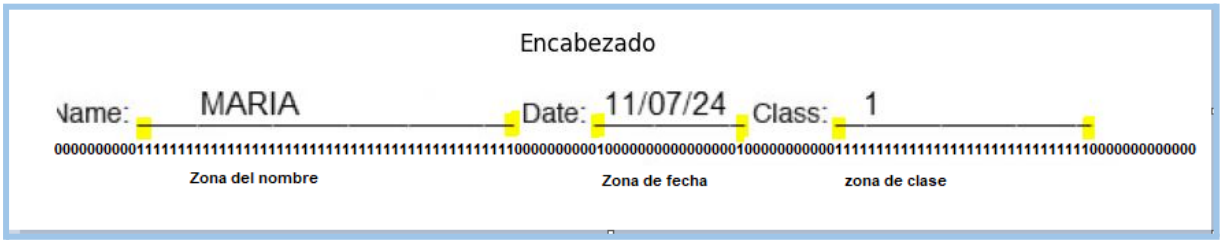
El paso siguiente fue el analizar lo completado por el evaluado, para ello realizamos dos estrategias diferenciadas, una para la zona del encabezado y otra para cada una de las áreas de respuestas

Análisis general del encabezado del encabezado .

Habiendo ya diferenciado el área en donde se encuentra el encabezado lo que que buscamos ahora es identificar las áreas en donde el alumno a completado su nombre, la fecha y el curso, para eso lo que hacemos en la imagen binarizada detectar la posición de la fila que contiene la línea que hace de “soporte” a los datos ingresados.
Una vez que tenemos esa línea lo que hacemos es recorrerla .. y ver los cambios de ceros y unos bajo el precepto donde hay “1” hay línea donde hay “0” hay corte de línea por lo cual detectando esos cambios podemos saber en donde se produce el principio y el fin de cada

línea, sabemos porque vimos los exámenes que la primer zona definida será la del nombre, la segunda la de la fecha y la tercera la correspondiente al campo clase.

Representación de como se diferenció las zonas perteneciente a cada campo



Estrategia general para procesar Nombre, fecha, y Clase

Habiendo segmentado estas zonas y teniendo las reglas de validación , la estrategia que se usó en cada uno de los casos es analizar la zona, poner parámetros para definir cantidad de componentes conectados, para eliminar el ruido, por un lado, y para eliminar la misma línea, la idea es quedarnos con los componentes conectados que representan caracteres

Validaciones

Una vez detectado los componentes conectados como así los espacios en cada tipo de dato se cruzan con la reglas de validación a los fines de resolver si cumplen con las reglas de validación, obteniendo una respuesta por consola de la situación

```
--- Examen: TP_1/examen_4.png ---
Name:  OK
Date:  MAL
Class: OK
```

Análisis de las respuestas obtenidas

Resuelto en el paso anterior la validación del encabezado, nos dedicamos a validar las respuestas de la persona examinada, para ello primero recurrimos otra vez a l detectar la zona de respuesta utilizando la línea de soporte del campo a completar, utilizando el mismo método con lo que detectamos la zona de respuesta con los campos de nombre, fecha , y clase.

En base a la línea soporte determinamos un área donde esperamos encontrar la respuesta, aplicamos reglas de validación generales para determinar primero si es una respuesta formalmente válida, en base a dos parámetros por un lado que la respuesta exista y por otro lado que no haya más de una respuesta para cada pregunta. Si pasa este filtro lo que hacemos es basarnos en la idea de contornos para detectar cuál fue la respuesta, y en forma subsidiaria con la detección de líneas

Detección de la letra

Bajo la premisa que las respuestas válidas solamente pueden ser A, B, C,D usamos estas regla de clasificación

1- Si la imagen analizada tiene dos huecos internos es una B

2- Si la imagen tiene cero huecos internos es una C

3- Si la imagen tiene un hueco interno, analizamos si tiene una línea recta vertical si la tiene es una D

4- Si no cumple con ninguno de los requisitos anteriores es una A

En un primer momento utilizamos otra estrategia para diferencias la A de la D que consistía en determinar la caja contenedora de la letra, y analizar el ancho y el alto, bajo la premisa que en los casos la A era más alta que ancha, logramos resultados consistentes pero la estrategia de detectar la línea recta de la D al final nos pareció más robusta y que dependía menos de la tipografía y más de la morfología general de cada uno de las letras involucradas.

Corregir el Examen

Teniendo las pautas de corrección, de exámen y habiendo capturado cada uno de los componentes de interés que lo componen, lo que procedemos es validar los datos y capturar las respuestas correctas e incorrectas por comparación con el patrón de corrección , y con eso emitir una imagen con con el resultado de cada uno de los alumnos

Reporte por consola de los resultados relevados en cada uno exámen - caso examen 4 -

```
--- Examen: TP_1/examen_4.png ---
Name:  OK
Date:  MAL
Class: OK
Pregunta  1: MAL  (detectada: B, correcta: C)
Pregunta  2: MAL  (detectada: C, correcta: B)
Pregunta  3: MAL  (detectada: D, correcta: A)
Pregunta  4: MAL  (detectada: B, correcta: D)
Pregunta  5: MAL  (detectada: A, correcta: B)
Pregunta  6: MAL  (detectada: A, correcta: B)
Pregunta  7: MAL  (detectada: C, correcta: A)
Pregunta  8: MAL  (detectada: D, correcta: B)
Pregunta  9: MAL  (detectada: B, correcta: D)
Pregunta 10: MAL  (detectada: A, correcta: D)
Resultado: DESAPROBADO (0/10 correctas)
```

Reporte visual de los resultados obtenidos por cada estudiante

REFLEXIONES FINALES AL PROBLEMA 2:

Durante la realización del trabajo nro 2 , hemos incorporado estrategias de segmentación en función de la existencia de líneas tanto verticales como horizontales.
En base a las líneas detectadas también hemos definido un área de interés para análisis posteriores.
También hemos utilizado herramientas para encontrar componentes conectados y la forma de contar la cantidad y su distribución en el espacio.
Otra herramienta importante para la solución ha sido la utilización de detección de contornos, como así su conteo que nos ayudó a interpretar y correlacionar las zonas de las imágenes relevadas con caracteres pertenecientes del alfabeto mediante el análisis de sus características morfológicas.